

클린 코드 관점의 테스트 코드

3-3 ~ 3-4

'읽히는 테스트'와 '충분히 검증하는 테스트' 정리

CONTENTS

오늘 함께 살펴볼 두 가지 이야기

01

3-3

테스트 코드도 클린 코드로 작성하기

이름 짓기 · Given-When-Then · 단일 검증 목표

02

3-4

커버리지를 고려한 테스트 코드 작성하기

다양한 케이스 · 라인 커버리지 · 테스트 보완

SECTION 01

3-3

테스트 코드도 클린 코드로 작성하기

INTRO

왜 테스트 코드도 '클린'해야 할까?

핵심

테스트 코드도 하나의 코드입니다.
'클린하게' 유지할 의무가 있어요.

유지 보수 시 테스트 코드도 함께 수정해야 함
코드 스멜이 많고 이해하기 어려우면 수정 부담 ↑
구현 코드와 마찬가지로 가독성이 매우 중요

이번 절에서 다룰 3가지

- 01 의도가 명확한 이름 짓기
- 02 Given-When-Then 구조
- 03 단일 검증 목표 + Assertions 최소화

3 - 3 · ①

의도가 명확하게 드러나는 이름 짓기

파일명 규칙

테스트 대상 파일 이름 + **Test.java**

EXAMPLE

구현 파일

Calculator.java



테스트 파일

CalculatorTest.java

3 - 3 · ①

함수명, 내부 로직 없이도 의도가 보여야 합니다.

BEFORE

userTest()

```
@Test
void userTest() {
    User user = new User(
        1, "test", "Kim", "M", 29, true);
    boolean result =
        userMgmtSystem.registerUser(user);
    assertEquals(true, result);
}
```

어떤 부분을 테스트하는지 의도가 안 보임

AFTER

의도가 드러나는 이름

```
// ① 영문 + 길지만 명확하게
successRegisterUserWhenGivenAllParam()

// ② 한글 + 스네이크 케이스
성공_활성화계정생성_모든속성포함()
```

'성공/실패 + 무엇을 + 어떤 조건에서'가 드러남

Given-When-Then

구조로 작성하기

3 단계 구조

Given

테스트 시작 전
조건·초기 상태·데이터 준비

When

테스트 대상 함수를
실제로 실행

Then

예상값과 결과값을 비교해
성공 여부 검증

CODE

주석으로 단계 구분 → 더 명확

```
@Test
void successRegister
    InactiveUserWhenGivenAllParam() {

    // Given - User 생성
    User user = normalUser;
    user.setActive(false);

    // When - 해당 User 등록
    Boolean result =
        userMgmtSystem.registerUser(user);

    // Then - 실행 결과 확인
    assertEquals(true, result);
}
```

하나의 검증 목표를 갖고 Assertions 함수 최소로 사용하기

원칙

01

한 함수, 한 검증 목표

여러 검증을 섞으면 어떤 구현 코드를 검증하려는지 파악 어려움
→ 결합도 ↑, 유지 보수 ↓

02

Assertions 함수는 1개 정도

한 가지 검증 목표에 1개 권장
여러 결괏값 검증이 꼭 필요할 때만 2개 이상 사용

TIP @BeforeAll로 사전 데이터 준비를 분리하고, 객체 전체를 한 번에 비교해 Assertions 사용을 최소화

SECTION 02

3-4

커버리지를 고려한 테스트 코드 작성하기

커버리지란 무엇일까요?

DEFINITION

작성한 테스트 코드가 실제로 구현 코드를 얼마나 실행하는지를
백분율로 나타내는 지표

이상적인 테스트 코드는 ...

01 정상 케이스

성공 시나리오

02 실패 케이스

예상 가능한 실패

03 예외 케이스

발생 가능한 예외

→ 다양한 케이스를 포괄할수록 라인 커버리지도 자연스럽게 ↑

3 - 4 · ①

다양한 케이스를 고려해 작성하기

제약 조건

회원 가입 시 Login과 PW 입력은 필수다.

STEP 1

테스트 코드 추가 (실패 예상)

```
@Test
void 실패_계정생성_Login값이없는경우() {
    User user = new User(100, "1234",
        null, "M", 29, true);

    Exception exception = assertThrows(
        Exception.class, () -> {
            userMgmtSystem.registerUser(user);
        });
}
```

STEP 2

구현 코드 보완 → 테스트 성공

```
boolean registerUser(User user) {
    if (user.login == null
        || user.login.isBlank()
        || user.pw == null
        || user.pw.isBlank()) {
        throw new NullPointerException(
            "Login값 또는 PW값 누락");
    }
    return users.add(user);
}
```

3 - 4 · ②

라인 커버리지로 시스템 커버 비율 확인하기

라인 커버리지란

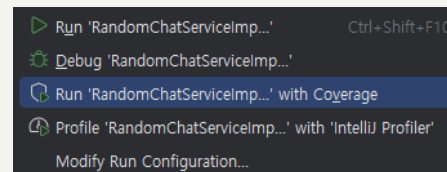
실제 실행된 라인 수
÷ 전체 구현 코드 라인 수

인텔리제이 실행

[Run] → Run '테스트 코드 파일명' with Coverage

권장

라인 커버리지 **80% 이상**



실전 흐름 · 19세 미만 가입 제한 추가 시

BEFORE

86%

19세 미만 예외 처리
로직이 미검증

↓ 나이 검증 테스트 함수 추가

AFTER

93%

예외 처리 로직까지
검증됨

WRAP - UP

정리하며 두 절을 한 줄로

3-3

테스트도 클린 코드

- **파일명:** 대상 파일명 + Test.java
- **함수명:** 내부 안 봐도 의도가 보이게
- **구조:** Given-When-Then
- **목표:** 한 함수에 하나, Assertions 최소

3-4

커버리지 고려

- **케이스:** 정상·실패·예외 모두 고려
- **관계:** 구현 ↔ 테스트는 상호 보완 (TDD 유사)
- **도구:** Run with Coverage로 누락 확인
- **권장:** 라인 커버리지 80% 이상